

Natural Language Processing 2

Feed Forward Neural Networks For Tabular Data

Fatemeh Dehrouyeh



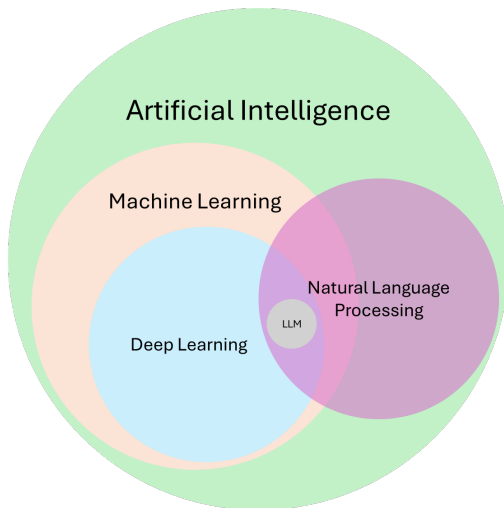
Contents

- 1 Introduction
- 2 Activation Functions
- 3 Backpropagation
 - Forward Pass
 - Compute the Loss Gradient Using the Chain Rule
 - Weight Update Using Gradient Descent
- 4 Loss Functions
 - Mean Squared Error
 - Cross-Entropy
- 5 Gradient Descent Update Strategies
- 6 Lab
 - Regression
 - Classification

Current Section

- 1 Introduction
- 2 Activation Functions
- 3 Backpropagation
- 4 Loss Functions
- 5 Gradient Descent Update Strategies
- 6 Lab

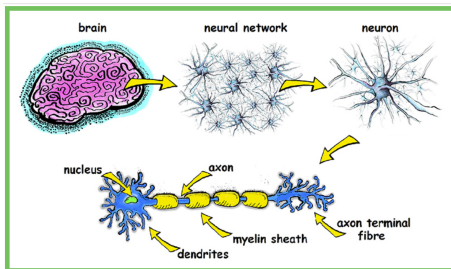
Artificial Intelligence and Its Subfields



Introduction to Neural Networks I

Neural Networks are computational models inspired by the human brain, consisting of interconnected nodes (neurons) that process and learn from data.

The Brain's Neural Network



The brain consists of numerous neurons interconnected in a complex neural network. Each neuron has a cell body, which houses the nucleus, and two types of extensions: dendrites and an axon. Dendrites receive incoming electrical signals, while the axon carries electrical signals away from the cell toward other neurons.¹

¹<https://kids.frontiersin.org/articles/10.3389/frym.2021.560631>

Red Wine Quality Dataset

Consider the "Red Wine Quality" dataset from Kaggle (**Source:** [Link](#)):

fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality
7.4	0.7	0	1.9	0.076	11	34	0.9978	3.51	0.56	9.4	5
7.8	0.88	0	2.6	0.098	25	67	0.9968	3.2	0.68	9.8	5
7.8	0.76	0.04	2.3	0.092	15	54	0.997	3.26	0.65	9.8	5
11.2	0.28	0.56	1.9	0.075	17	60	0.998	3.16	0.58	9.8	6
7.4	0.7	0	1.9	0.076	11	34	0.9978	3.51	0.56	9.4	5
7.4	0.66	0	1.8	0.075	13	40	0.9978	3.51	0.56	9.4	5
7.9	0.6	0.06	1.6	0.069	15	59	0.9964	3.3	0.46	9.4	5
7.3	0.65	0	1.2	0.065	15	21	0.9946	3.39	0.47	10	7
7.8	0.58	0.02	2	0.073	9	18	0.9968	3.36	0.57	9.5	7
7.5	0.5	0.36	6.1	0.071	17	102	0.9978	3.35	0.8	10.5	5
6.7	0.58	0.08	1.8	0.097	15	65	0.9959	3.28	0.54	9.2	5
7.5	0.5	0.36	6.1	0.071	17	102	0.9978	3.35	0.8	10.5	5
5.6	0.615	0	1.6	0.089	16	59	0.9943	3.58	0.52	9.9	5
7.8	0.61	0.29	1.6	0.114	9	29	0.9974	3.26	1.56	9.1	5
8.9	0.62	0.18	3.8	0.176	52	145	0.9986	3.16	0.88	9.2	5
8.9	0.62	0.19	3.9	0.17	51	148	0.9986	3.17	0.93	9.2	5
8.5	0.28	0.56	1.8	0.092	35	103	0.9969	3.3	0.75	10.5	7
8.1	0.56	0.28	1.7	0.368	16	56	0.9968	3.11	1.28	9.3	5
7.4	0.59	0.08	4.4	0.086	6	29	0.9974	3.38	0.5	9	4

This dataset is designed for predicting wine quality based on numerical physicochemical features. The target variable, **quality**, can be used for regression if we predict the quality score, or for classification if we convert the scores into categories such as low, medium, and high quality.

**Now let's discuss neural
networks in AI!**

Neural Network Categories

● Feed-Forward Neural Networks (FFNN):

- FFNNs are a foundational type of neural network where information moves in one direction, from input to output.
- They often use fully connected layers, where each neuron connects to neurons in the next layer.
- They are suitable for structured/tabular data and tasks such as regression and classification.

● Recurrent Neural Networks (RNN):

- RNNs are designed for sequential data.
- They use recurrent connections that allow information from previous steps to influence later predictions.
- They are used for tasks such as time-series prediction, speech recognition, and text processing.

● Convolutional Neural Networks (CNN):

- CNNs use convolutional layers to capture local patterns and spatial features.
- They are commonly used for image processing, object detection, and video analysis.
- They can also be applied to other structured grid-like data.

Key Insight: Different neural network architectures are designed for different data structures, such as tabular data, sequences, and images.

Key Components of Neural Networks

Neural networks have several main components:

- **Input Layer:**

- Receives the wine characteristics, such as acidity, sugar, pH, alcohol content, and sulphates.

- **Hidden Layers:**

- Intermediate layers where computation occurs, learning patterns and relationships between the wine characteristics and its quality.

- **Output Layer:**

- Produces the predicted wine quality score based on its physicochemical characteristics.

- **Weights and Biases:**

- Adjustable parameters optimized during training to reduce the difference between the predicted and actual wine quality scores.

Neural Networks Components (Regression)

This is a Multi-Layer Perceptron (MLP):

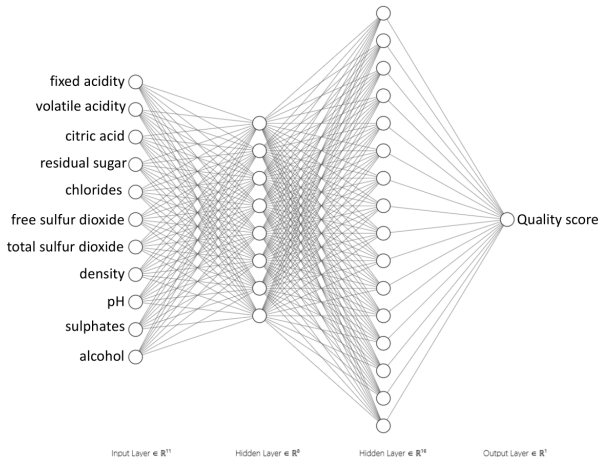
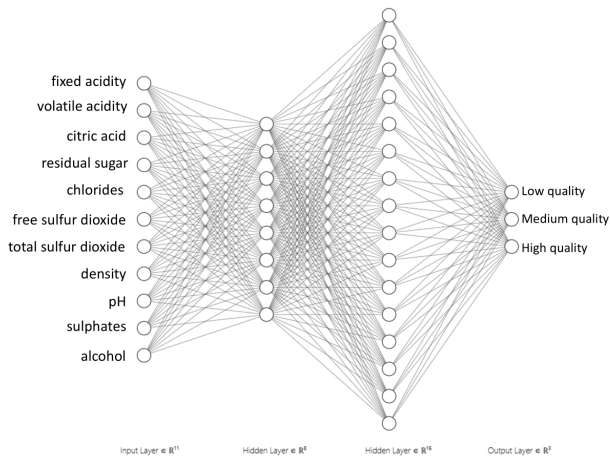
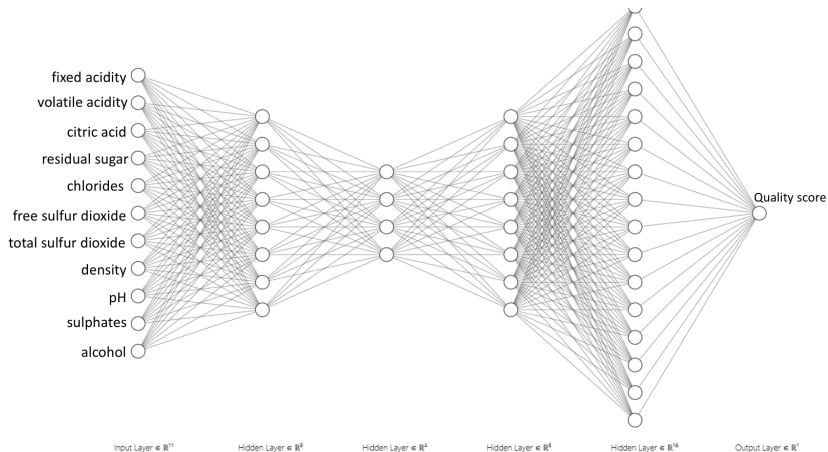


Figure created using NN-SVG.

Neural Networks Components (Classification)



Hyperparameter Optimizations



Feedforward Neural Network (FFNN) vs. Multilayer Perceptron (MLP)

- **Feedforward Neural Network (FFNN):**

- A broad class of neural networks where information flows only in one direction:

Input $\rightarrow$ Hidden Layers $\rightarrow$ Output

- There are no feedback or recurrent connections.
- Examples include MLPs and many standard CNN architectures.

- **Multilayer Perceptron (MLP):**

- A specific type of feedforward neural network.
- Consists of an input layer, one or more hidden layers, and an output layer.
- Neurons in consecutive layers are typically **fully connected**.
- Uses nonlinear activation functions such as ReLU or sigmoid.

- **Key Difference:**

- **FFNN** describes the direction of information flow, while **MLP** describes a specific fully connected feedforward architecture.

Benefits of Neural Networks

- **Adaptability:** Neural networks can learn complex relationships from data without requiring explicit programming.
- **Non-Linearity:** Can model complex, non-linear patterns in data.
- **Scalability:** Scales well with larger datasets and more complex architectures.

Challenges of Neural Networks

- **Computational Cost:** Requires significant computational resources for training large models.
- **Overfitting:** Prone to memorizing training data, reducing generalization to unseen data.
- **Data Dependence:** Requires large, high-quality datasets for optimal performance.
- **Interpretability:** Often described as **black-box** models, making them difficult to interpret.
- **Hyperparameter Tuning:** Performance depends on carefully choosing architecture and parameters, which can be time-consuming.

Takeaway: Neural networks are powerful tools for solving complex problems but come with trade-offs in interpretability, computational cost, and data requirements.

Current Section

- 1 Introduction
- 2 Activation Functions**
- 3 Backpropagation
- 4 Loss Functions
- 5 Gradient Descent Update Strategies
- 6 Lab

Introduction to Activation Functions

Activation Functions introduce non-linearity into neural networks, allowing them to model complex patterns and relationships in data.

- Activation functions determine if a neuron should be **activated**, influencing how the network learns from data.
- Without activation functions, a neural network would behave like a linear model, limiting its ability to capture complex patterns.

Common Activation Functions

Several activation functions are widely used, each with unique properties suited to different tasks.

- **Linear**
- **Sigmoid**
- **Tanh (Hyperbolic Tangent)**
- **ReLU (Rectified Linear Unit)**
- **Leaky ReLU**
- **Softmax**

Key Consideration

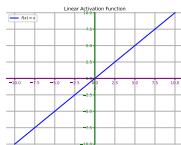
The choice of activation function impacts network performance, training speed, and the ability to model complex relationships.

Linear Activation Function

The **Linear** activation function computes the output as a direct proportion of the input.

$$f(x) = x$$

- Suitable for regression tasks, such as predicting stock prices or housing values.
- **Output Range:** $-\infty$ to ∞ , depending on the input.
- **Advantages:** Simple and effective for tasks where the output is not bounded.
- **Limitations:** Cannot capture non-linear relationships; all layers behave as a single linear transformation when combined.

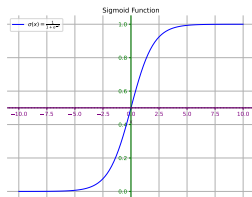


Sigmoid Activation Function

The **Sigmoid** function compresses input values into a range between 0 and 1.

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

- Suitable for binary classification tasks, such as determining if a loan is high or low risk.
- **Output Range:** 0 to 1, representing probabilities.
- **Limitations:** Prone to vanishing gradients for large values of x , which slows training.

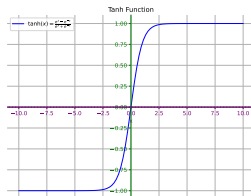


Tanh Activation Function

The **Tanh** function scales input values between -1 and 1, providing a wider range than sigmoid.

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

- Suitable for models where both negative and positive values are meaningful.
- **Output Range:** -1 to 1, centered at zero.
- **Benefit:** Reduces issues with vanishing gradients compared to sigmoid.

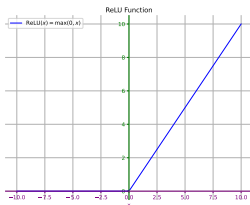


ReLU Activation Function

ReLU outputs the input directly if it's positive; otherwise, it outputs zero.

$$f(x) = \max(0, x)$$

- Widely used in hidden layers due to simplicity and efficiency, enabling faster training.
- **Output Range:** 0 to infinity for positive values.
- **Benefit:** Reduces issues with vanishing gradients.
- **Limitation:** Can cause **dead neurons** if too many values become zero, especially in noisy data.

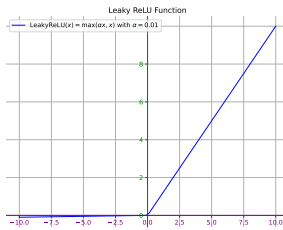


Leaky ReLU Activation Function

Leaky ReLU introduces a small slope for negative inputs, addressing the **dead neuron** issue.

$$f(x) = \begin{cases} x & x > 0 \\ \alpha x & x \leq 0 \end{cases}$$

- **Output Range:** Negative values are retained, though scaled down by a factor (e.g., $\alpha = 0.01$).
- **Benefit:** Prevents dead neurons and enhances model stability.



Softmax Activation Function

Softmax normalizes output values into probabilities, useful for multi-class classification.

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

- Typically used in the output layer for risk categories, such as low, medium, or high.
- **Output Range:** 0 to 1, with the sum of outputs equal to 1, representing class probabilities.
- **Benefit:** Provides a clear probabilistic interpretation.

Softmax Activation Function - Numerical Example I

Scenario: Wine Quality Classification

- A neural network is predicting the probability that a red wine belongs to one of three quality classes: **Low Quality, Medium Quality, and High Quality.**
- The raw output logits from the network are:

$$z_{\text{Low}} = 0.5, \quad z_{\text{Medium}} = 2.0, \quad z_{\text{High}} = 1.0$$

Softmax Formula:

$$P(i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Softmax Activation Function - Numerical Example II

Step-by-Step Calculation:

- Exponentiate the logits:

$$e^{z_{\text{Low}}} = e^{0.5} \approx 1.65,$$

$$e^{z_{\text{Medium}}} = e^{2.0} \approx 7.39,$$

$$e^{z_{\text{High}}} = e^{1.0} \approx 2.72$$

- Sum of exponentials:

$$\text{Sum} = 1.65 + 7.39 + 2.72 = 11.76$$

Softmax Activation Function - Numerical Example III

- Compute class probabilities:

$$P(\text{Low}) = \frac{1.65}{11.76} \approx 0.14,$$

$$P(\text{Medium}) = \frac{7.39}{11.76} \approx 0.63,$$

$$P(\text{High}) = \frac{2.72}{11.76} \approx 0.23$$

Interpretation:

- **Low Quality:** 14% predicted probability.
- **Medium Quality:** 63% predicted probability.
- **High Quality:** 23% predicted probability.
- Since **Medium Quality** has the highest probability, the neural network predicts:

Medium Quality

Choosing the Right Activation Function

Selecting the appropriate activation function depends on the specific application.

- **Sigmoid:**

- Use for binary classification.

- **ReLU (Rectified Linear Unit):**

- Default choice for hidden layers in deep networks.

- **Softmax:**

- Ideal for multi-class classification problems.
- Outputs probabilities that sum to 1.

- **Tanh:**

- Centered around zero, often used in hidden layers.

Key Insight

The choice of activation function depends on the problem type, data, and network architecture. Start with ReLU for hidden layers and Softmax/Sigmoid for output layers.

Current Section

1 Introduction

2 Activation Functions

3 Backpropagation

- Forward Pass
- Compute the Loss Gradient Using the Chain Rule
- Weight Update Using Gradient Descent

4 Loss Functions

5 Gradient Descent Update Strategies

6 Lab

Introduction to Backpropagation

Backpropagation is an optimization algorithm used in neural networks to minimize the error by adjusting weights.

- Backpropagation calculates the gradient of the loss function with respect to each weight, propagating errors backward through the network.
- This process allows the network to learn from mistakes and iteratively reduce prediction errors.

Steps in Backpropagation

The backpropagation algorithm involves the following steps:

① Forward Pass:

- Calculate the network's output by passing the input through each layer using the current weights and biases.
- Compute the loss by comparing the output to the target value.

② Compute the Loss Gradient Using the Chain Rule:

- Use the **chain rule** to calculate the gradient of the loss function with respect to each weight and bias in the network.

③ Backward Pass and Weight Update Using Gradient Descent:

- Propagate the error backward through the network to compute gradients for all layers.
- Use the **gradient descent** algorithm to iteratively update weights and biases, minimizing the loss function.

Chain Rule in Backpropagation I

What is the Chain Rule?

- The chain rule is a fundamental concept in calculus used to compute the derivative of a composite function.
- It is the key mathematical principle enabling backpropagation to compute gradients for weights and biases efficiently.

General Formula: If a function L depends on w through intermediate variables z and y :

$$L = f(y), \quad y = g(z), \quad z = h(w)$$

Then the chain rule states:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w}$$

Chain Rule in Backpropagation II

- During the **Compute the Loss Gradient** step, the chain rule is applied layer by layer.
- Gradients flow from the output layer to the input layer, adjusting weights and biases to minimize the loss.
- Each layer's weights and biases are updated based on these gradients in the **Weight and Bias Update** step.

Key Insight: The chain rule enables the backpropagation algorithm to compute gradients for deep neural networks, making training efficient and scalable.

Gradient Descent Algorithm

What is Gradient Descent?

- Gradient descent is an optimization algorithm used to minimize the loss function in a neural network.
- It updates weights and biases iteratively, using the gradients computed by backpropagation.

Gradient Descent Update Rule: For a weight w and bias b :

$$w \leftarrow w - \alpha \cdot \frac{\partial \text{Loss}}{\partial w}, \quad b \leftarrow b - \alpha \cdot \frac{\partial \text{Loss}}{\partial b}$$

Where:

- α : Learning rate (controls the step size).
- $\frac{\partial \text{Loss}}{\partial w}$: Gradient of the loss with respect to w .
- $\frac{\partial \text{Loss}}{\partial b}$: Gradient of the loss with respect to b .

What is Gradient Descent? I

The main idea is:

Move the parameters in the direction that decreases the loss.

Suppose the loss depends on one weight w , so $L = L(w)$.

The derivative $\frac{\partial L}{\partial w}$ tells us the slope of the loss curve with respect to w .

The weight is updated using $w_{\text{new}} = w_{\text{old}} - \alpha \frac{\partial L}{\partial w}$, where α is the **learning rate**.

Interpretation:

- If the slope is positive, gradient descent moves w to the left.
- If the slope is negative, gradient descent moves w to the right.
- The goal is to reach a point where the loss is as small as possible.

What is Gradient Descent? II

In Neural Networks, the loss depends on many parameters:

$$L = L(w_1, w_2, \dots, w_n)$$

So we update all weights using the gradient:

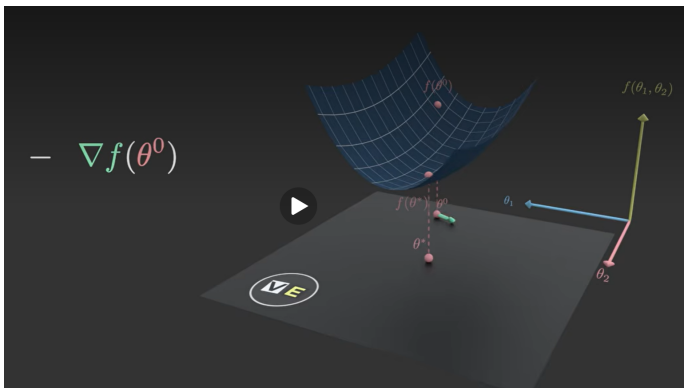
$$w_{\text{new}} = w_{\text{old}} - \alpha \nabla L$$

where

$$\nabla L = \begin{bmatrix} \frac{\partial L}{\partial w_1} \\ \frac{\partial L}{\partial w_2} \\ \vdots \\ \frac{\partial L}{\partial w_n} \end{bmatrix}.$$

Backpropagation computes the gradients, gradient descent uses them to update the parameters.

What is Gradient Descent? III



Watch this video

Steps in Backpropagation - Numerical Example

Assumptions:

Input Values:

$$x_1 = 1, \quad x_2 = 2$$

Weights:

$$w_1 = w_2 = w_3 = w_4 = w_5 = w_6 = 0.5$$

Biases:

$$b_{\text{hidden}} = 0, \quad b_{\text{output}} = 0$$

Activation Function:

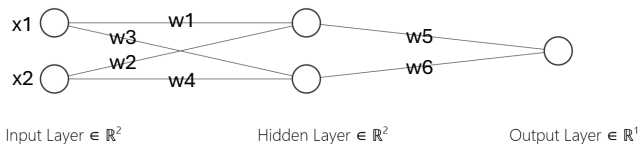
$$f(x) = x \quad (\text{Linear Activation Function})$$

Target Output:

$$y_{\text{target}} = 3$$

Learning Rate:

$$\alpha = 0.1$$



Current Subsection

1 Introduction

2 Activation Functions

3 Backpropagation

- **Forward Pass**

- Compute the Loss Gradient Using the Chain Rule

- Weight Update Using Gradient Descent

4 Loss Functions

5 Gradient Descent Update Strategies

6 Lab

1. Forward Pass

- Calculate the output of each hidden neuron:

$$z_{\text{hidden}_1} = w_1 \cdot x_1 + w_2 \cdot x_2 + b_{\text{hidden}_1} = 0.5 \cdot 1 + 0.5 \cdot 2 + 0 = 1.5$$

$$z_{\text{hidden}_2} = w_3 \cdot x_1 + w_4 \cdot x_2 + b_{\text{hidden}_2} = 0.5 \cdot 1 + 0.5 \cdot 2 + 0 = 1.5$$

- Calculate the output of the network:

$$y_{\text{output}} = w_5 \cdot z_{\text{hidden}_1} + w_6 \cdot z_{\text{hidden}_2} + b_{\text{output}} = 0.5 \cdot 1.5 + 0.5 \cdot 1.5 + 0 = 1.5$$

- Compute the loss (Mean Squared Error):

$$\text{Loss} = \frac{1}{2}(y_{\text{target}} - y_{\text{output}})^2 = \frac{1}{2}(3 - 1.5)^2 = 1.125$$

Current Subsection

1 Introduction

2 Activation Functions

3 Backpropagation

- Forward Pass
- **Compute the Loss Gradient Using the Chain Rule**
- Weight Update Using Gradient Descent

4 Loss Functions

5 Gradient Descent Update Strategies

6 Lab

2. Compute the Loss Gradient Using the Chain Rule I

Recall the loss function:

$$L = \frac{1}{2}(y_{\text{target}} - y_{\text{output}})^2$$

and

$$y_{\text{output}} = w_5 z_{\text{hidden}_1} + w_6 z_{\text{hidden}_2} + b_{\text{output}}.$$

Since

$$y_{\text{output}} = 1.5, \quad y_{\text{target}} = 3,$$

we first compute

$$\frac{\partial L}{\partial y_{\text{output}}} = \frac{\partial}{\partial y_{\text{output}}} \left[\frac{1}{2}(y_{\text{target}} - y_{\text{output}})^2 \right].$$

Using the chain rule,

$$\frac{\partial L}{\partial y_{\text{output}}} = \frac{1}{2}(2)(y_{\text{target}} - y_{\text{output}})(-1).$$

Therefore,

$$\frac{\partial L}{\partial y_{\text{output}}} = y_{\text{output}} - y_{\text{target}} = 1.5 - 3 = -1.5.$$

$$\frac{\partial L}{\partial y_{\text{output}}} = -1.5$$

2. Compute the Loss Gradient Using the Chain Rule II

1. Gradient with respect to w_5

Using the chain rule,

$$\frac{\partial L}{\partial w_5} = \frac{\partial L}{\partial y_{\text{output}}} \cdot \frac{\partial y_{\text{output}}}{\partial w_5}.$$

We already know

$$\frac{\partial L}{\partial y_{\text{output}}} = -1.5.$$

Now,

$$\frac{\partial y_{\text{output}}}{\partial w_5} = \frac{\partial}{\partial w_5} (w_5 z_{\text{hidden}_1} + w_6 z_{\text{hidden}_2} + b_{\text{output}}).$$

Therefore,

$$\frac{\partial y_{\text{output}}}{\partial w_5} = z_{\text{hidden}_1} = 1.5.$$

Hence,

$$\frac{\partial L}{\partial w_5} = (-1.5)(1.5) = -2.25.$$

$$\frac{\partial L}{\partial w_5} = -2.25$$

2. Compute the Loss Gradient Using the Chain Rule III

For w_6 : Same pattern

$$\frac{\partial L}{\partial w_6} = \frac{\partial L}{\partial y_{\text{output}}} \cdot \frac{\partial y_{\text{output}}}{\partial w_6}$$

with

$$\frac{\partial y_{\text{output}}}{\partial w_6} = z_{\text{hidden}_2} = 1.5.$$

Therefore,

$$\frac{\partial L}{\partial w_6} = (-1.5)(1.5) = -2.25$$

2. Compute the Loss Gradient Using the Chain Rule IV

2. Gradient with respect to w_1

Recall

$$z_{\text{hidden}_1} = w_1 x_1 + w_2 x_2 + b_{\text{hidden}_1}.$$

The path from w_1 to the loss is

$$w_1 \rightarrow z_{\text{hidden}_1} \rightarrow y_{\text{output}} \rightarrow L.$$

Using the chain rule,

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial y_{\text{output}}} \cdot \frac{\partial y_{\text{output}}}{\partial z_{\text{hidden}_1}} \cdot \frac{\partial z_{\text{hidden}_1}}{\partial w_1}.$$

First,

$$\frac{\partial L}{\partial y_{\text{output}}} = -1.5.$$

Next,

$$\frac{\partial y_{\text{output}}}{\partial z_{\text{hidden}_1}} = \frac{\partial}{\partial z_{\text{hidden}_1}} (w_5 z_{\text{hidden}_1} + w_6 z_{\text{hidden}_2} + b_{\text{output}}) = w_5 = 0.5.$$

Also,

$$\frac{\partial z_{\text{hidden}_1}}{\partial w_1} = \frac{\partial}{\partial w_1} (w_1 x_1 + w_2 x_2 + b_{\text{hidden}_1}) = x_1 = 1.$$

2. Compute the Loss Gradient Using the Chain Rule V

Therefore,

$$\frac{\partial L}{\partial w_1} = (-1.5)(0.5)(1) = -0.75.$$

$$\boxed{\frac{\partial L}{\partial w_1} = -0.75}$$

For w_3 : Same pattern

The path is

$$w_3 \rightarrow z_{\text{hidden}_2} \rightarrow y_{\text{output}} \rightarrow L.$$

Thus,

$$\frac{\partial L}{\partial w_3} = \frac{\partial L}{\partial y_{\text{output}}} \cdot \frac{\partial y_{\text{output}}}{\partial z_{\text{hidden}_2}} \cdot \frac{\partial z_{\text{hidden}_2}}{\partial w_3}.$$

Using

$$\frac{\partial y_{\text{output}}}{\partial z_{\text{hidden}_2}} = w_6 = 0.5, \quad \frac{\partial z_{\text{hidden}_2}}{\partial w_3} = x_1 = 1,$$

we obtain

$$\boxed{\frac{\partial L}{\partial w_3} = (-1.5)(0.5)(1) = -0.75}$$

2. Compute the Loss Gradient Using the Chain Rule VI

3. Gradient with respect to w_2

Using the chain rule,

$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial y_{\text{output}}} \cdot \frac{\partial y_{\text{output}}}{\partial z_{\text{hidden}_1}} \cdot \frac{\partial z_{\text{hidden}_1}}{\partial w_2}.$$

We already have

$$\frac{\partial L}{\partial y_{\text{output}}} = -1.5$$

and

$$\frac{\partial y_{\text{output}}}{\partial z_{\text{hidden}_1}} = w_5 = 0.5.$$

Now,

$$\frac{\partial z_{\text{hidden}_1}}{\partial w_2} = \frac{\partial}{\partial w_2} (w_1 x_1 + w_2 x_2 + b_{\text{hidden}_1}) = x_2 = 2.$$

Therefore,

$$\frac{\partial L}{\partial w_2} = (-1.5)(0.5)(2) = -1.5.$$

$$\frac{\partial L}{\partial w_2} = -1.5$$

2. Compute the Loss Gradient Using the Chain Rule VII

For w_4 : Same pattern

$$\frac{\partial L}{\partial w_4} = \frac{\partial L}{\partial y_{\text{output}}} \cdot \frac{\partial y_{\text{output}}}{\partial z_{\text{hidden}_2}} \cdot \frac{\partial z_{\text{hidden}_2}}{\partial w_4}.$$

Using

$$\frac{\partial y_{\text{output}}}{\partial z_{\text{hidden}_2}} = w_6 = 0.5, \quad \frac{\partial z_{\text{hidden}_2}}{\partial w_4} = x_2 = 2,$$

we obtain

$$\frac{\partial L}{\partial w_4} = (-1.5)(0.5)(2) = -1.5$$

2. Compute the Loss Gradient Using the Chain Rule VIII

4. Gradient with respect to b_{hidden_1}

Using the chain rule,

$$\frac{\partial L}{\partial b_{\text{hidden}_1}} = \frac{\partial L}{\partial y_{\text{output}}} \cdot \frac{\partial y_{\text{output}}}{\partial z_{\text{hidden}_1}} \cdot \frac{\partial z_{\text{hidden}_1}}{\partial b_{\text{hidden}_1}}.$$

We have

$$\frac{\partial L}{\partial y_{\text{output}}} = -1.5$$

and

$$\frac{\partial y_{\text{output}}}{\partial z_{\text{hidden}_1}} = w_5 = 0.5.$$

Now,

$$\frac{\partial z_{\text{hidden}_1}}{\partial b_{\text{hidden}_1}} = \frac{\partial}{\partial b_{\text{hidden}_1}} (w_1 x_1 + w_2 x_2 + b_{\text{hidden}_1}) = 1.$$

Therefore,

$$\frac{\partial L}{\partial b_{\text{hidden}_1}} = (-1.5)(0.5)(1) = -0.75.$$

$$\frac{\partial L}{\partial b_{\text{hidden}_1}} = -0.75$$

2. Compute the Loss Gradient Using the Chain Rule IX

For b_{hidden_2} : Same pattern

$$\frac{\partial L}{\partial b_{\text{hidden}_2}} = \frac{\partial L}{\partial y_{\text{output}}} \cdot \frac{\partial y_{\text{output}}}{\partial z_{\text{hidden}_2}} \cdot \frac{\partial z_{\text{hidden}_2}}{\partial b_{\text{hidden}_2}}.$$

Using

$$\frac{\partial y_{\text{output}}}{\partial z_{\text{hidden}_2}} = w_6 = 0.5, \quad \frac{\partial z_{\text{hidden}_2}}{\partial b_{\text{hidden}_2}} = 1,$$

we obtain

$$\frac{\partial L}{\partial b_{\text{hidden}_2}} = (-1.5)(0.5)(1) = -0.75$$

2. Compute the Loss Gradient Using the Chain Rule X

5. Gradient with respect to b_{output}

Using the chain rule,

$$\frac{\partial L}{\partial b_{\text{output}}} = \frac{\partial L}{\partial y_{\text{output}}} \cdot \frac{\partial y_{\text{output}}}{\partial b_{\text{output}}}.$$

We know

$$\frac{\partial L}{\partial y_{\text{output}}} = -1.5.$$

Now,

$$\frac{\partial y_{\text{output}}}{\partial b_{\text{output}}} = \frac{\partial}{\partial b_{\text{output}}} (w_5 z_{\text{hidden}_1} + w_6 z_{\text{hidden}_2} + b_{\text{output}}) = 1.$$

Therefore,

$$\frac{\partial L}{\partial b_{\text{output}}} = (-1.5)(1) = -1.5.$$

$$\boxed{\frac{\partial L}{\partial b_{\text{output}}} = -1.5}$$

2. Compute the Loss Gradient Using the Chain Rule XI

Summary of Gradients

$$\frac{\partial L}{\partial w_1} = -0.75, \quad \frac{\partial L}{\partial w_2} = -1.5, \quad \frac{\partial L}{\partial w_3} = -0.75$$

$$\frac{\partial L}{\partial w_4} = -1.5, \quad \frac{\partial L}{\partial w_5} = -2.25, \quad \frac{\partial L}{\partial w_6} = -2.25$$

$$\frac{\partial L}{\partial b_{\text{hidden}_1}} = -0.75, \quad \frac{\partial L}{\partial b_{\text{hidden}_2}} = -0.75, \quad \frac{\partial L}{\partial b_{\text{output}}} = -1.5$$

Current Subsection

1 Introduction

2 Activation Functions

3 Backpropagation

- Forward Pass
- Compute the Loss Gradient Using the Chain Rule
- **Weight Update Using Gradient Descent**

4 Loss Functions

5 Gradient Descent Update Strategies

6 Lab

3. Weight Update Using Gradient Descent I

New weights and biases after one iteration of backpropagation.

Learning Rate (α): 0.1

- **Update w_5 :**

$$w_5 = w_5 - \alpha \cdot \frac{\partial \text{Loss}}{\partial w_5} = 0.5 - 0.1 \cdot (-2.25) = 0.725$$

- **Update w_6 :**

$$w_6 = w_6 - \alpha \cdot \frac{\partial \text{Loss}}{\partial w_6} = 0.5 - 0.1 \cdot (-2.25) = 0.725$$

3. Weight Update Using Gradient Descent II

- **Update w_1 :**

$$w_1 = w_1 - \alpha \cdot \frac{\partial \text{Loss}}{\partial w_1} = 0.5 - 0.1 \cdot (-0.75) = 0.575$$

- **Update w_2 :**

$$w_2 = w_2 - \alpha \cdot \frac{\partial \text{Loss}}{\partial w_2} = 0.5 - 0.1 \cdot (-1.5) = 0.65$$

- **Update w_3 :**

$$w_3 = w_3 - \alpha \cdot \frac{\partial \text{Loss}}{\partial w_3} = 0.5 - 0.1 \cdot (-0.75) = 0.575$$

- **Update w_4 :**

$$w_4 = w_4 - \alpha \cdot \frac{\partial \text{Loss}}{\partial w_4} = 0.5 - 0.1 \cdot (-1.5) = 0.65$$

3. Weight Update Using Gradient Descent III

- Update b_{hidden_1} :

$$b_{\text{hidden}_1} = b_{\text{hidden}_1} - \alpha \cdot \frac{\partial \text{Loss}}{\partial b_{\text{hidden}_1}} = 0 - 0.1 \cdot (-0.75) = 0.075$$

- Update b_{hidden_2} :

$$b_{\text{hidden}_2} = b_{\text{hidden}_2} - \alpha \cdot \frac{\partial \text{Loss}}{\partial b_{\text{hidden}_2}} = 0 - 0.1 \cdot (-0.75) = 0.075$$

Updated Values:

- $w_1 = 0.575$, $w_2 = 0.65$, $w_3 = 0.575$, $w_4 = 0.65$
- $w_5 = 0.725$, $w_6 = 0.725$
- $b_{\text{hidden}_1} = 0.075$, $b_{\text{hidden}_2} = 0.075$

**How having Sigmoid Function
as activation function can cause
vanishing gradient problem?**

Vanishing Gradient Problem in Sigmoid

The sigmoid activation function is defined as:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Its derivative (slope) is:

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

- Around $x = 0$, the sigmoid is at its **steepest**:

$$\sigma(0) = 0.5 \quad \Rightarrow \quad \sigma'(0) = 0.5(1 - 0.5) = 0.25$$

- For large positive x : $\sigma(x) \rightarrow 1 \quad \Rightarrow \quad \sigma'(x) \rightarrow 0$

The curve becomes almost flat.

- For large negative x : $\sigma(x) \rightarrow 0 \quad \Rightarrow \quad \sigma'(x) \rightarrow 0$

The curve also becomes almost flat.

The problem is that in the saturated regions, its **slope (derivative) approaches zero**. Deep networks face difficulty in effective training when using sigmoid, as gradients diminish across layers.

Terminologies

From One Training Step to an Epoch

What we just did:

- We performed a **forward pass** to compute the prediction.
- We calculated the **loss**.
- We used **backpropagation** to compute the gradients.
- We then use these gradients to update the weights and biases.

Important: The numerical example we just completed represents the learning process for one training observation. In practice, a neural network repeats this process many times across the training dataset.

What is an Epoch?

Epoch

An **epoch** is one complete pass through the entire training dataset.

For example, if the training dataset contains 1,000 observations:

1 Epoch = the model has processed all 1,000 observations once

Batch Size, Iterations, and Epochs

How is the training data processed?

- **Batch Size:**

- The number of training observations processed before the model updates its weights.

- **Mini-Batch:**

- A small subset of the training dataset used for one weight update.

- **Iteration:**

- One forward pass + one backward pass + one parameter update using one batch.

- **Epoch:**

- One complete pass through the entire training dataset.

Example

Suppose we have 1,000 training observations and use a batch size of 100:

$$\text{Number of iterations per epoch} = \frac{1000}{100} = 10$$

Therefore,

$$1 \text{ Epoch} = 10 \text{ Iterations}$$

Current Section

- 1 Introduction
- 2 Activation Functions
- 3 Backpropagation
- 4 Loss Functions**
 - Mean Squared Error
 - Cross-Entropy
- 5 Gradient Descent Update Strategies
- 6 Lab

What are some common loss function?

Loss Functions

The **Loss Function** measures the difference between predicted and actual values. Common loss functions include:

- **Mean Squared Error (MSE)** for regression tasks.
- **Cross-Entropy Loss** for classification tasks, such as predicting credit risk categories.

Gradient of the Loss Function

The gradient of the loss function indicates the direction and magnitude of changes needed to reduce error.

$$\text{Gradient} = \frac{\partial \text{Loss}}{\partial \text{Weights}} \quad (1)$$

Current Subsection

- 1 Introduction
- 2 Activation Functions
- 3 Backpropagation
- 4 Loss Functions**
 - Mean Squared Error
 - Cross-Entropy
- 5 Gradient Descent Update Strategies
- 6 Lab

MSE Loss Function

Mean Squared Error (MSE):

- Definition: Measures the average squared difference between the predicted and actual values in regression tasks.
- Formula:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Where:

- y_i : Actual value
- $\hat{y}_i$: Predicted value
- N : Number of data points (samples)

Current Subsection

- 1 Introduction
- 2 Activation Functions
- 3 Backpropagation
- 4 Loss Functions**
 - Mean Squared Error
 - **Cross-Entropy**
- 5 Gradient Descent Update Strategies
- 6 Lab

Cross-Entropy Loss Function

Cross-Entropy Loss:

- Definition: Measures the difference between two probability distributions, often used for classification tasks.
- **Formula (Binary Classification):**

$$\text{Cross-Entropy Loss} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

Where:

- y_i : Actual label (0 or 1)
- $\hat{y}_i$: Predicted probability ($0 \leq \hat{y}_i \leq 1$)
- N : Number of data points

Cross-Entropy Loss Function (Multi-Class)

Cross-Entropy Loss:

- Definition: Measures the difference between the true class labels and the predicted probability distribution over multiple classes.
- **Formula (Multi-Class Classification):**

$$\text{Cross-Entropy Loss} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(\hat{y}_{i,c})$$

Where:

- $y_{i,c}$: One-hot encoded actual label for class c of data point i (1 if class c is correct, otherwise 0).
- $\hat{y}_{i,c}$: Predicted probability for class c of data point i ($0 \leq \hat{y}_{i,c} \leq 1$).
- N : Number of data points.
- C : Total number of classes.

How It Works: If the model predicts a high probability for the correct class, the loss is small.

Deriving Binary Cross-Entropy from NLL

For binary classification, the model predicts:

$$\hat{y} = P(y = 1 | x)$$

For one observation, the probability assigned to the true label is:

$$P(y | \hat{y}) = \hat{y}^y (1 - \hat{y})^{1-y}$$

The negative log-likelihood is:

$$\text{NLL} = -\log(P(y | \hat{y}))$$

Substituting the binary probability:

$$\text{NLL} = -\log(\hat{y}^y (1 - \hat{y})^{1-y})$$

Using log rules:

$$\text{NLL} = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

For a dataset of N observations:

$$\text{Binary Cross-Entropy} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

Binary cross-entropy is the negative log-likelihood of a Bernoulli model. Minimizing it encourages the model to assign high probability to the true class.

Why Negative Log-Likelihood?

For a dataset with N observations, the likelihood of the observed data is:

$$L = p_1 p_2 p_3 \cdots p_N$$

where each p_i is the probability that the model assigns to the correct outcome for observation i .

The log turns multiplication into addition:

$$\log L = \log p_1 + \log p_2 + \cdots + \log p_N$$

This is much easier to optimize numerically.

Why the negative sign?

For a good prediction:

$$p = 0.9 \quad \Rightarrow \quad -\log(0.9) \approx 0.105$$

For a poor prediction:

$$p = 0.1 \quad \Rightarrow \quad -\log(0.1) \approx 2.303$$

Key Idea Good predictions produce a small negative log-likelihood, while poor confident predictions produce a large negative log-likelihood.

Therefore, minimizing NLL is equivalent to encouraging the model to assign high probability to the correct outcomes.

Current Section

- 1 Introduction
- 2 Activation Functions
- 3 Backpropagation
- 4 Loss Functions
- 5 Gradient Descent Update Strategies**
- 6 Lab

When to Update the Gradient?

Batch, Mini-Batch, and Stochastic Gradient Descent

Main Idea

Gradient descent updates the model parameters in the direction that reduces the loss:

$$w_{\text{new}} = w_{\text{old}} - \alpha g$$

where:

- α is the learning rate,
- g is the gradient used for the current update.

The three main approaches differ in **how many training observations are used to compute g before updating the parameters.**

Three Main Approaches

1 Batch Gradient Descent

- Uses the **entire training dataset** before each parameter update.
- Batch size:

$$B = N$$

where N is the total number of training observations.

2 Mini-Batch Gradient Descent

- Uses a **subset** of the training dataset before each update.
- Typical batch sizes:

$$B = 32, 64, 128, \dots$$

3 Stochastic Gradient Descent (SGD)

- Uses **one observation** before each parameter update.
- Batch size:

$$B = 1$$

$$B = 1 \longrightarrow \text{Mini-Batch} \longrightarrow B = N$$

SGD

Mini-Batch

Batch GD

How Are Multiple Observations Treated?

Suppose the batch contains B observations.

For each observation i , the network produces a prediction $\hat{y}_i$ and an individual loss L_i .

For example, using squared error:

$$L_i = \frac{1}{2} (y_i - \hat{y}_i)^2$$

The individual losses are typically **averaged**:

$$L_{\text{batch}} = \frac{1}{B} \sum_{i=1}^B L_i$$

Backpropagation then computes the gradient of this average loss:

$$\frac{\partial L_{\text{batch}}}{\partial w} = \frac{\partial}{\partial w} \left(\frac{1}{B} \sum_{i=1}^B L_i \right)$$

Batch Gradient Descent I

If the training dataset contains N observations, the loss is

$$L_{\text{batch}} = \frac{1}{N} \sum_{i=1}^N L_i$$

and the gradient is

$$g = \frac{1}{N} \sum_{i=1}^N \frac{\partial L_i}{\partial w}$$

The parameter update is

$$w_{\text{new}} = w_{\text{old}} - \alpha \left(\frac{1}{N} \sum_{i=1}^N \frac{\partial L_i}{\partial w} \right)$$

Update frequency:

One parameter update per epoch

Batch Gradient Descent II

Advantages:

- Uses information from the entire dataset.
- Produces a stable and relatively smooth gradient estimate.

Disadvantages:

- Can be computationally expensive for large datasets.
- Requires processing the entire dataset before each update.
- Can require substantial memory.

Stochastic Gradient Descent (SGD) I

SGD uses only one observation i at a time.
Therefore,

$$L_{\text{SGD}} = L_i$$

and

$$g = \frac{\partial L_i}{\partial w}$$

The parameter update becomes

$$w_{\text{new}} = w_{\text{old}} - \alpha \frac{\partial L_i}{\partial w}$$

Update frequency:

One parameter update after every observation

If there are N observations:

N updates per epoch

Stochastic Gradient Descent (SGD) II

Advantages:

- Very frequent updates.
- Each update is computationally inexpensive.
- The noise in the gradient can sometimes help optimization move through saddle points or flat regions.

Disadvantages:

- Individual gradients can be noisy.
- The loss may fluctuate considerably during training.
- Modern hardware is usually used less efficiently than with mini-batches.

Mini-Batch Gradient Descent I

Suppose each mini-batch contains B observations, where

$$1 < B < N.$$

The mini-batch loss is

$$L_{\text{batch}} = \frac{1}{B} \sum_{i=1}^B L_i.$$

The gradient is

$$g = \frac{1}{B} \sum_{i=1}^B \frac{\partial L_i}{\partial w}$$

The parameter update is therefore

$$w_{\text{new}} = w_{\text{old}} - \alpha \left(\frac{1}{B} \sum_{i=1}^B \frac{\partial L_i}{\partial w} \right)$$

Mini-Batch Gradient Descent II

Update frequency:

If there are N observations and batch size B ,

$$\text{Iterations per epoch} \approx \frac{N}{B}$$

More precisely, if the last incomplete batch is retained:

$$\text{Iterations per epoch} = \left\lceil \frac{N}{B} \right\rceil$$

Advantages:

- More stable gradients than SGD.
- More frequent updates than full-batch gradient descent.
- Efficient for parallel computation on GPUs.
- Most commonly used approach for training neural networks.

Disadvantages:

- Batch size is another hyperparameter to choose.
- Gradients still contain some sampling noise.

Example: $N = 1000$ Training Observations

Batch Gradient Descent

$$\frac{B}{1000} = 1$$

Therefore:

1 update per epoch

Mini-Batch Gradient Descent

Suppose

$$B = 100.$$

Then

$$\frac{1000}{100} = 10.$$

Therefore:

10 updates per epoch

Stochastic Gradient Descent

$$B = 1$$

so

$$\frac{1000}{1} = 1000.$$

Therefore:

1000 updates per epoch

Summary

Method	Batch Size	Data per Update	Updates/Epoch
SGD	1	1 observation	N
Mini-Batch	B	B observations	$\lceil N/B \rceil$
Batch GD	N	Entire dataset	1

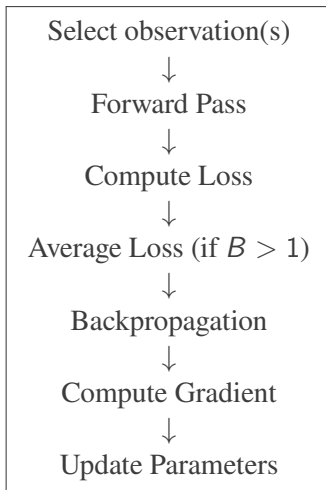
Key Idea

All three methods use the same gradient-descent update rule:

$$w_{\text{new}} = w_{\text{old}} - \alpha g.$$

The difference is **how much training data is used to calculate the loss and gradient before each parameter update.**

Summary



What are advanced variants of gradient descent algorithm?

Advanced Types of Gradient Descent

Advanced Gradient Descent Variants:

- **Momentum:** Accelerates SGD by adding a fraction of the previous update vector.
- **Nesterov Accelerated Gradient (NAG):** Improves momentum by looking ahead at the future position.
- **Adaptive Learning Rate Methods:**
 - **AdaGrad:** Adjusts learning rates based on historical gradients.
 - **RMSProp:** Combines AdaGrad with exponential decay of past gradients.
 - **Adam:** Combines momentum and RMSProp for robust optimization.

Key Distinction

These algorithms modify the gradient computation and learning rate to improve optimization.

Current Section

- 1 Introduction
- 2 Activation Functions
- 3 Backpropagation
- 4 Loss Functions
- 5 Gradient Descent Update Strategies
- 6 Lab**
 - Regression
 - Classification

Current Subsection

- 1 Introduction
- 2 Activation Functions
- 3 Backpropagation
- 4 Loss Functions
- 5 Gradient Descent Update Strategies
- 6 Lab**
 - Regression
 - Classification

Regression—Preprocessing

Considered the mentioned dataset for wine quality:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping

# 1. Load dataset
df = pd.read_csv("winequality-red.csv")

# Check column names
print(df.head())
print(df.columns)

# 2. Split features and target
X = df.drop("quality", axis=1)
y = df["quality"]

# 3. Train, validation, test split
X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.30, random_state=42)

X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.50, random_state=42)

# 4. Standardize numerical features
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)
```

Regression—Building MLP

```
# 5. Build FFNN model
model = Sequential([
    Dense(64, activation="relu", input_shape=(X_train_scaled.shape[1],)),
    Dense(32, activation="relu"),
    Dense(16, activation="relu"),
    Dense(1, activation="linear")
])

# 6. Compile model
model.compile(optimizer="adam", loss="mse", metrics=["mae"])

# 7. Early stopping
early_stop = EarlyStopping(monitor="val_loss", patience=20, restore_best_weights=True)

# 8. Train model
history = model.fit(X_train_scaled, y_train, validation_data=(X_val_scaled, y_val),
                    epochs=200, batch_size=32, callbacks=[early_stop], verbose=1)

# 9. Evaluate model
y_pred = model.predict(X_test_scaled).flatten()

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("Test MAE:", mae)
print("Test MSE:", mse)
print("Test RMSE:", rmse)
print("Test R2:", r2)

# 10. Example predictions
results = pd.DataFrame({"Actual Quality": y_test.values, "Predicted Quality": y_pred})
print(results.head(10))
```

Regression—Printing first 10 results

```

35/35 ————— 0s 4ms/step - loss: 0.2493 - mae: 0.3866 - val_loss: 0.4109 - val_mae: 0.5151
Epoch 68/200
35/35 ————— 0s 4ms/step - loss: 0.2520 - mae: 0.3861 - val_loss: 0.3765 - val_mae: 0.4996
Epoch 69/200
35/35 ————— 0s 4ms/step - loss: 0.2421 - mae: 0.3776 - val_loss: 0.3846 - val_mae: 0.4961
Epoch 70/200
35/35 ————— 0s 4ms/step - loss: 0.2412 - mae: 0.3746 - val_loss: 0.4037 - val_mae: 0.5086
Epoch 71/200
35/35 ————— 0s 4ms/step - loss: 0.2537 - mae: 0.3866 - val_loss: 0.4011 - val_mae: 0.5045
Epoch 72/200
35/35 ————— 0s 6ms/step - loss: 0.2426 - mae: 0.3781 - val_loss: 0.3792 - val_mae: 0.4883
Epoch 73/200
35/35 ————— 0s 7ms/step - loss: 0.2556 - mae: 0.3904 - val_loss: 0.3982 - val_mae: 0.5115
Epoch 74/200
35/35 ————— 0s 6ms/step - loss: 0.2454 - mae: 0.3867 - val_loss: 0.3865 - val_mae: 0.4953
8/8 ————— 0s 15ms/step
Test MAE: 0.5001126527786255
Test MSE: 0.421212762594223
Test RMSE: 0.649009062027814
Test R2: 0.37340253591537476
Actual Quality Predicted Quality
0          6      5.700114
1          5      5.331021
2          7      5.715696
3          6      5.866632
4          7      6.458478
5          7      6.612134
6          6      6.103870
7          6      6.116304
8          5      5.838519
9          6      7.181866

```

Current Subsection

- 1 Introduction
- 2 Activation Functions
- 3 Backpropagation
- 4 Loss Functions
- 5 Gradient Descent Update Strategies
- 6 Lab**
 - Regression
 - **Classification**

Classification—Preprocessing

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping

# 1. Load dataset
df = pd.read_csv("winequality-red.csv")

# Check column names
print(df.head())
print(df.columns)

# 2. Convert quality score into classes
# 0 = Low quality
# 1 = Medium quality
# 2 = High quality
def quality_to_class(q):
    if q <= 5:
        return 0
    elif q == 6:
        return 1
    else:
        return 2

df["quality_class"] = df["quality"].apply(quality_to_class)

print(df[["quality", "quality_class"]].head())
print(df["quality_class"].value_counts())
```

Classification—Preprocessing

```
# 3. Split features and target
X = df.drop(["quality", "quality_class"], axis=1)
y = df["quality_class"]

# 4. Train, validation, test split
X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.30, random_state=42, stratify=y)

X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.50, random_state=42, stratify=y_temp)

# 5. Standardize numerical features
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)
```

Classification—Building MLP

```
# 6. Build FFNN classification model
model = Sequential([
    Dense(64, activation="relu", input_shape=(X_train_scaled.shape[1],)),
    Dropout(0.2),
    Dense(32, activation="relu"),
    Dense(16, activation="relu"),
    Dense(3, activation="softmax")
])

# 7. Compile model
model.compile(optimizer="adam", loss="sparse_categorical_crossentropy", metrics=["accuracy"])

# 8. Early stopping
early_stop = EarlyStopping(monitor="val_loss", patience=20, restore_best_weights=True)

# 9. Train model
history = model.fit(X_train_scaled, y_train, validation_data=(X_val_scaled, y_val),
                    epochs=200, batch_size=32, callbacks=[early_stop], verbose=1)

# 10. Evaluate model
test_loss, test_accuracy = model.evaluate(X_test_scaled, y_test, verbose=0)

print("Test Loss:", test_loss)
print("Test Accuracy:", test_accuracy)

# 11. Predictions
y_pred_prob = model.predict(X_test_scaled)
y_pred = np.argmax(y_pred_prob, axis=1)
```

Classification—Printing Result

```
print("Accuracy Score:", accuracy_score(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=["Low", "Medium", "High"]))

# 12. Example predictions
results = pd.DataFrame({
    "Actual Class": y_test.values,
    "Predicted Class": y_pred,
    "Low Probability": y_pred_prob[:, 0],
    "Medium Probability": y_pred_prob[:, 1],
    "High Probability": y_pred_prob[:, 2]
})

print(results.head(10))
```

Classification—Result

```

35/35 ————— 0s 4ms/step - accuracy: 0.7069 - loss: 0.6421 - val_accuracy: 0.6417 - val_loss: 0.7537
Epoch 48/200
35/35 ————— 0s 5ms/step - accuracy: 0.7140 - loss: 0.6347 - val_accuracy: 0.6458 - val_loss: 0.7455
Test Loss: 0.6738559603691101
Test Accuracy: 0.7166666388511658
8/8 ————— 0s 9ms/step
Accuracy Score: 0.7166666666666667

```

Confusion Matrix:

```

[[85 25  2]
 [23 67  6]
 [ 0 12 20]]

```

Classification Report:

	precision	recall	f1-score	support
Low	0.79	0.76	0.77	112
Medium	0.64	0.70	0.67	96
High	0.71	0.62	0.67	32
accuracy			0.72	240
macro avg	0.72	0.69	0.70	240
weighted avg	0.72	0.72	0.72	240

Actual Class	Predicted Class	Low Probability	Medium Probability	\
0	0	0.227607	0.620560	
1	1	0.139970	0.759063	
2	2	0.024483	0.420076	
3	2	0.088145	0.476814	
4	0	0.679226	0.316907	
5	0	0.503427	0.454057	
6	1	0.652987	0.336122	
7	0	0.592787	0.400157	
8	1	0.027747	0.304371	
9	1	0.236417	0.701873	

High Probability

0	0.151833
1	0.100966
2	0.555440
3	0.435041
4	0.003867
5	0.042516
6	0.010971
7	0.007137
8	0.667882
9	0.061710

Choosing the Correct Cross-Entropy Loss

The type of cross-entropy depends on the classification task and how the target labels are stored.

Binary Classification

For two classes, use:

```
binary_crossentropy
```

Example labels:

0, 1

Multiclass Classification

If labels are stored as integers:

cat = 0, dog = 1, bird = 2

use:

```
sparse_categorical_crossentropy
```

If labels are one-hot encoded:

cat = [1, 0, 0], dog = [0, 1, 0], bird = [0, 0, 1]

use:

```
categorical_crossentropy
```

Key Idea

The word **sparse** refers to the label format: integer class labels instead of one-hot vectors.

Macro, Micro, and Weighted Averages

Macro Average: compute the metric for each class separately, then take the simple average.

$$M_{\text{macro}} = \frac{1}{C} \sum_{c=1}^C M_c$$

Each class has equal importance, regardless of class size.

Micro Average: add the true positives, false positives, and false negatives across all classes first, then compute the metric.

$$P_{\text{micro}} = \frac{\sum_c TP_c}{\sum_c TP_c + \sum_c FP_c}$$

$$R_{\text{micro}} = \frac{\sum_c TP_c}{\sum_c TP_c + \sum_c FN_c}$$

Weighted Average: compute the metric for each class, then weight it by the number of true samples in that class.

$$M_{\text{weighted}} = \frac{\sum_{c=1}^C n_c M_c}{\sum_{c=1}^C n_c}$$

where n_c is the support of class c .

Congratulations!

Now you know how MLPs work!

